

Insider-Signal Trading System

Automated Trading Daemon, Strategy-Simulation Platform & Quantitative Backtesting Engine

Technical Architecture Documentation

Author: László Czap | Repository: Insiderlive | State: September 2026

1. Executive Summary

This document describes the design and implementation of a fully automated, long-running trading system built end-to-end from scratch: a Python daemon that ingests a public data feed, runs it through a multi-stage filtering and risk-sizing pipeline, executes and reconciles real broker orders, and exposes full operational control and observability through a chat-based interface. Alongside the live trading path, the codebase includes a parallel multi-strategy simulation framework used for ongoing signal-quality research, and a separate offline backtesting engine used for historical strategy validation.

The intent of this document is to describe the engineering of the system — architecture, data flow, persistence design, concurrency model, resilience patterns, testing, and deployment.

Project scope at a glance

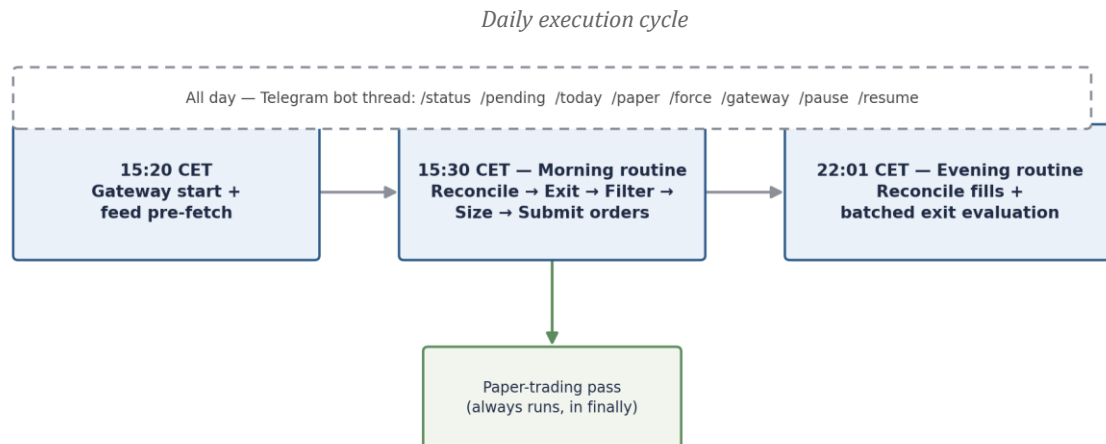
Aspect	Summary
Problem domain	Automated equity trading triggered by public insider-transaction filings (SEC Form 4 / OpenInsider)
Codebase size	16 Python modules at the application root, a dedicated pytest suite, containerized deployment
External systems	Interactive Brokers (order execution), Telegram (control & alerting), OpenInsider (web-scraped signal source), Yahoo Finance (market & fundamental data), Sentry (error monitoring)
Persistence	PostgreSQL, four independently-owned schema groups, hand-rolled idempotent migrations
Concurrency model	Multi-threaded daemon (scheduler thread + Telegram bot thread) with explicit lock-guarded shared state
Testing	pytest with boundary mocking (broker, DB, HTTP, market-data client) across 9 test modules
Deployment	Self-hosted on my Linux Ubuntu server, on private home network; Docker + docker-compose (Postgres + bot services), push-to-deploy via a self-hosted GitHub Actions runner on that same host
Adjacent subsystems	A 6-strategy × 2-sizing-variant paper-trading simulation engine, and a standalone historical backtesting/validation engine (Monte Carlo, walk-forward, grid search)

2. System Architecture & Data Flow

The system is built as a modular pipeline of single-responsibility services, wired together at runtime without a DI framework. `scheduler.py` acts as the composition root: it instantiates one shared broker connection object and passes it down to every component that needs it for the duration of the process.

2.1 Daily execution cycle

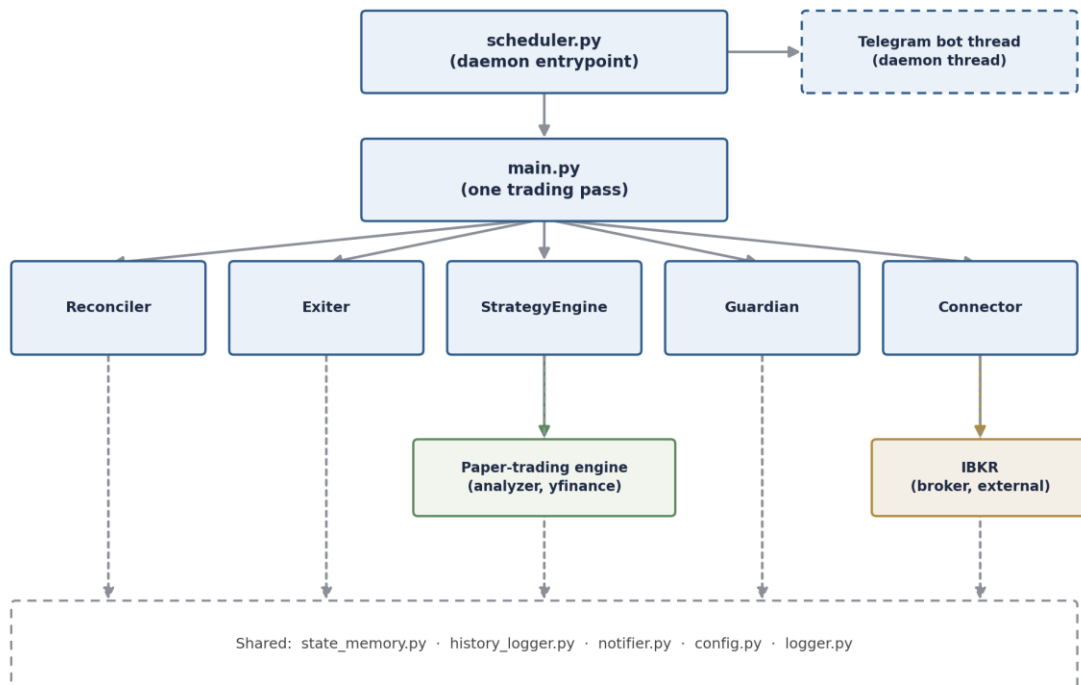
- **Scheduled startup (15:20 CET):** the daemon starts the IB Gateway process and pre-fetches the day's insider-filing feed into a local cache.
- **Morning routine (15:30 CET):** `Reconciler.sync_data()` reconciles broker state with the local database, `Exiter` evaluates time-based exits, `StrategyEngine` produces filtered entry candidates, `Guardian` sizes positions against available equity, and approved orders are submitted to the broker as bracket orders.
- A parallel, unconditional pass (wrapped in a `finally` block, so it always runs regardless of live-trading outcome) feeds the same raw, less-filtered signal stream into the paper-trading simulation engine.
- **Evening routine (22:01 CET):** `Reconciler.sync_data()` runs a second time (fills, partial fills, cancellations, equity snapshot), followed by a batched exit-evaluation pass across all open paper-trading positions.
- Throughout the day, a Telegram bot thread exposes command-driven introspection and control (`/status`, `/pending`, `/today`, `/paper`, `/connection`, `/force`, `/gateway`, `/pause`, `/resume`) against the same shared state.



2.2 Module dependency direction

`scheduler.py` sits at the top, owning `main.py`'s trading cycle and the paper-trading engine. `main.py` orchestrates `reconciler`, `engine`, `guardian`, `exiter`, `state_memory` and `notifier` for a single pass. All modules depend on `config.py` for validated settings and `logger.py` for structured logging. `analyzer.py` is a leaf dependency consumed only by the paper-trading engine — the live entry pipeline defines fundamental-quality filter parameters but does not currently invoke the fundamental analyzer, a deliberate scope decision.

Figure 2 — Module architecture & data flow



2.3 Module reference

Module	Responsibility
<code>scheduler.py</code>	Process entrypoint; job scheduling, Telegram bot thread, gateway lifecycle, Sentry init
<code>main.py</code>	Orchestrates one trading pass: reconcile → exit → filter → size → submit orders
<code>config.py</code>	Pydantic-settings model: typed, validated, immutable configuration loaded from <code>.env</code>
<code>connector.py</code>	Broker integration (<code>ib_insync</code>): connection lifecycle, bracket orders, fallback logic
<code>engine.py</code>	Signal filtering pipeline: regime filter, seasonal filter, dedup, pre-drop overlay
<code>feeder.py</code>	Web-scrapes the insider-filing feed; same-day cache; structural-change alerting
<code>guardian.py</code>	Position sizing against available equity; fractional/whole-share allocation
<code>exiter.py</code>	Time-based (max-hold) exit evaluation; SL/TP are native broker bracket legs
<code>reconciler.py</code>	Broker ↔ database consistency layer: fills, partial fills, cancellations, equity snapshot
<code>state_memory.py</code>	Live trading state store: open positions, pending orders, cash balance
<code>history_logger.py</code>	Append-only trade ledger, daily equity snapshots, equity-curve report generation
<code>analyzer.py</code>	Piotroski F-Score fundamental analysis with a 24h fetch cache and audit log
<code>paper_logger.py</code>	Multi-strategy, multi-portfolio simulation and research engine (see § 7)
<code>notifier.py</code>	Telegram push-notification client used as an operational alerting channel
<code>models.py</code>	Shared typed contracts (<code>TradeSignal</code> , <code>SignalAction</code>) passed between pipeline stages
<code>logger.py</code>	Rotating file + console logging factory shared across the application

3. External Integrations

System	Library / Protocol	Integration pattern
Interactive Brokers	ib_insync (TWS API)	Persistent connection shared across threads under an RLock; event-driven callbacks detect market-data-farm disconnects; bracket orders with automatic fractional→whole-share fallback on broker rejection
Telegram	pyTelegramBotAPI (interactive) + raw HTTPS calls (alerts)	Two separate paths: a polling command bot in its own daemon thread, and a lightweight REST wrapper used across nearly every module as a general operational-alerting channel
OpenInsider	requests + BeautifulSoup	Fixed-schema HTML table scraping with a same-day file cache; scrape failures are classified by cause (HTTP error, timeout, missing table, zero rows) and each triggers a distinct alert
Yahoo Finance	yfinance	Market regime filter (SPY vs. 200-day moving average), price fallback when the broker cannot quote a symbol, and financial-statement retrieval for fundamental scoring
Sentry	sentry-sdk with a logging integration	Error-level log records are promoted to Sentry events; info-level records are attached as breadcrumbs; wired into the daemon entrypoint only
PostgreSQL	psycopg2 (writes/DDDL) + SQLAlchemy (pandas reads)	Sole datastore; four modules independently own their schema and run idempotent startup migrations

4. Persistence Layer

All state is stored in PostgreSQL, migrated from an earlier SQLite-based design. Rather than a shared ORM or a central schema module, each of the four stateful modules owns and migrates its own tables, following one consistent idiom: CREATE TABLE IF NOT EXISTS at startup, an additive ALTER TABLE ... ADD COLUMN IF NOT EXISTS migration step for schema evolution, INSERT ... ON CONFLICT DO UPDATE upserts for idempotent writes, and rollback-on-exception around every write path.

Owner module	Tables	Purpose
state_memory.py	positions, pending_orders, balance	Live trading state: current holdings, in-flight orders, cash
history_logger.py	trades_log, daily_equity	Append-only fill ledger and daily equity snapshots for reporting
analyzer.py	ticker_raw_cache, analysis_log	24h TTL cache of raw financial statements; audit trail of every F-Score computed
paper_logger.py	signals, paper_positions, paper_portfolios, portfolio_allocations	Full research dataset: every raw signal, per-strategy simulated positions, per-portfolio capital simulations, and the allocation join table linking them

5. Concurrency & Scheduling Model

The daemon deliberately uses a minimal, two-thread model rather than a fully asynchronous architecture, even though the broker client library depends on an underlying async event loop.

- **Main thread:** a cooperative polling loop (the ``schedule`` library) running jobs at mixed cadences — one-minute keepalive checks, two-minute price-cache refreshes, five-second polls for user-triggered actions, and fixed daily trading-cycle times.
- **Daemon thread:** the Telegram bot's long-polling loop, isolated so a Telegram outage can never block trading logic; it maintains its own reconnect-with-backoff loop.
- **Cross-thread coordination uses a small, consistent pattern rather than fine-grained locking of business logic:** a shared broker connection guarded by a re-entrant lock, a live-price cache guarded by its own lock, and boolean request flags (force-run, gateway-start) that the Telegram thread sets and the main loop polls and clears every five seconds — avoiding any scenario where trading logic executes concurrently with itself.
- **Startup catch-up:** if the process restarts mid-trading-day (e.g. after a container redeploy) while market hours and connectivity conditions are already met, the daemon immediately runs one trading pass instead of waiting for the next scheduled slot.
- **Weekend housekeeping:** the keepalive job proactively disconnects the broker session outside weekdays to free the account's two-factor-authenticated session slot for mobile use.

6. Resilience & Error-Handling Patterns

Pattern	What it solves
Layered price sourcing	position sizing and the simulation engine both fall back from a live broker quote to a market-data-provider historical close, so a temporary broker data-feed outage never fully blocks a pricing decision.
Fractional-share order recovery	bracket-order submission detects the broker's specific fractional-share rejection error codes and automatically resubmits the identical order structure at a whole-share quantity, rather than failing the trade outright.
Broker/database reconciliation	a dedicated module diffs live broker positions and orders against the local database on every cycle, correctly handling full fills, partial fills (re-queued with the remaining quantity), and cancellations/rejections — with every branch wrapped in try/except/finally so a broker-side hiccup can never crash the daemon.
Scraper failure classification	the signal feeder distinguishes HTTP failure, timeout, missing page structure, and zero-rows-parsed as distinct failure modes, each raising a specific alert — functioning as an early-warning canary for upstream page-layout changes.
Duplicate-alert throttling	repeated authentication or connection failures are alerted once, not on every retry, avoiding notification spam during extended outages.
Operational alerting as a first-class channel	nearly every module pushes structured alerts through the same notification channel used for trade confirmations — disconnects, partial fills, cancelled orders, and scraper breakage are all surfaced in real time, not only written to a log file.

7. Multi-Strategy Simulation & Research Engine

A secondary module, decoupled from the live order path but fed the same raw signal stream, runs six pluggable decision rules against a single shared feature fetch per signal and simulates their resulting positions across twelve capital-allocation variants — used for ongoing signal-quality research without risking live capital. This is a supporting subsystem for future testing and data collecting for proofing alpha.

8. Historical Backtesting & Validation Engine

Separate from the live daemon, a dedicated offline research codebase was built to validate the signal strategy against historical data before any capital was committed. The description below covers the engine's technical design; exact signal-quality thresholds and financial performance figures are intentionally omitted.

8.1 Engine design

- **Custom event-driven, mark-to-market simulation rather than a vectorized backtest:** positions are opened and closed following simulated capital constraints, so the engine can only take a trade when it would genuinely have had the cash to do so — rather than assuming unlimited capital — which makes fill-order path-dependency meaningful to test.
- Bisection-indexed historical price series for $O(\log n)$ date lookups, avoiding repeated linear DataFrame filtering across thousands of simulated trade evaluations.
- **Strict no-look-ahead construction:** every signal is evaluated only against information that would have been available at that point in time.

8.2 Validation methodology

- Two-factor Monte Carlo simulation (randomized fill order and randomized signal drop-out, run at multiple sample-size levels) used to test whether results were an artifact of trade sequencing or of which subset of signals happened to be sampled.
- Anchored walk-forward validation across multiple rolling in-sample/out-of-sample windows, used to check for in-sample overfitting rather than relying on a single train/test split.
- A large combinatorial grid search over strategy parameters, scored with a risk-adjusted composite metric and cross-checked using partial-dependence (marginal) averaging and coefficient-of-variation analysis — to distinguish genuinely robust parameter plateaus from narrow, overfit spikes.
- **A data-quality bug found and corrected mid-analysis:** an unintended universe-selection restriction (the backtest was inadvertently limited to a narrower ticker universe than intended) was identified, root-caused, and fixed by rebuilding the dataset from an unrestricted scrape — a concrete example of catching and correcting a subtle selection bias before drawing conclusions from the results.

9. Testing Strategy

The application is covered by a pytest suite spanning nine modules (analyzer, engine, exiter, feeder, guardian, history_logger, notifier, reconciler, state_memory), following a consistent mock-at-the-boundary approach: the broker client, database layer, HTTP calls, and market-data client are all replaced with mocks so that business logic — position-sizing math, business-day exit thresholds, market-regime and drawdown calculations, HTML-parsing logic, and reconciliation diff/merge logic — is exercised deterministically, without live network or broker access.

10. CI/CD & Containerization

10.1 Deployment pipeline

A GitHub Actions workflow runs on push to the main branch (or manual dispatch) as two dependent jobs. A test job first checks out the repository on a standard GitHub-hosted runner, installs dependencies, and runs the full pytest suite; a deploy job then runs on a self-hosted runner only if the test job succeeded (needs: test). Deploy performs an authenticated fetch (temporarily rewriting the git remote to embed a short-lived token, then restoring the plain HTTPS remote so no token is persisted in git configuration), resets the working tree to the latest commit, and rebuilds and restarts the containerized service via docker compose.

10.2 Containerization

- Dockerfile: python:3.11-slim base, with Xvfb/X11 packages installed to run the broker's Java desktop gateway headlessly, plus the toolchain needed for chart rendering; the container's entrypoint runs the scheduler daemon directly.
- docker-compose: two services — a PostgreSQL database with a named persistent volume, and the bot service built from the local Dockerfile, run in host networking mode so it can reach the broker gateway's local ports and the gateway can in turn reach the host's display/session state. Secrets are injected via an env file; the broker's own installation directories and session-persistence files are bind-mounted rather than baked into the image.
- Hosting: both the docker-compose stack and the self-hosted GitHub Actions runner run on the author's own Linux Ubuntu server on a private home network, rather than a cloud provider — the deploy job connects directly to this host to rebuild and restart the container.

11. Configuration & Secrets Management

All tunables are centralized in a single pydantic-settings model loaded from an .env file: typed, range-validated (e.g. bounded percentages and scores), immutable once loaded, and tolerant of unrelated environment variables. Configuration is grouped by concern — infrastructure/observability (error-monitoring DSN, database URL, credentials), broker connection, strategy/filter toggles, and notification credentials — with an .env.example template enumerating every expected variable without values. Secrets, generated logs, runtime caches, and legacy database artifacts are all excluded from version control via .gitignore.

12. Notable Engineering Decisions

- Clean mid-project storage migration from SQLite to PostgreSQL across four independently-owned schema modules, executed without downtime, using idempotent additive migrations rather than a destructive cutover.
- **Defense-in-depth around a fundamentally brittle upstream dependency (HTML scraping):** failure modes are classified and alerted individually rather than caught generically, turning scraper breakage into an immediately visible operational event instead of a silent data gap.
- Framework-free, idempotent schema evolution applied consistently across every persistence module, keeping deployments migration-safe without introducing a separate migration tool.
- A strategy-pattern simulation engine that fetches expensive shared inputs once per signal and fans them out to independently pluggable, side-effect-free decision functions — a design that scales to new strategy variants with minimal code change and no redundant API usage.

- Deliberate resilience engineering for a real, observed third-party API quirk (fractional-share order rejection) rather than assuming happy-path broker behavior.
- Treating an operational chat channel as a first-class alerting surface across almost every module, giving real-time visibility into an otherwise-unattended daemon process.

13. Known Limitations & Future Improvements

The items below are architectural trade-offs rather than defects.

- Per-module hand-rolled schema migrations work well at the current schema size but would benefit from consolidation into a single migration tool if the schema continues to grow.
- The signal source is a single scraped website with a fixed HTML layout; the existing failure-alerting reduces risk but a structural layout change would still require a manual parser update.
- The test suite currently covers business logic at the unit level; it does not yet include an integration test against a real (e.g. containerized) PostgreSQL instance, which would catch SQL-dialect or schema-migration mistakes that unit-level mocking cannot.